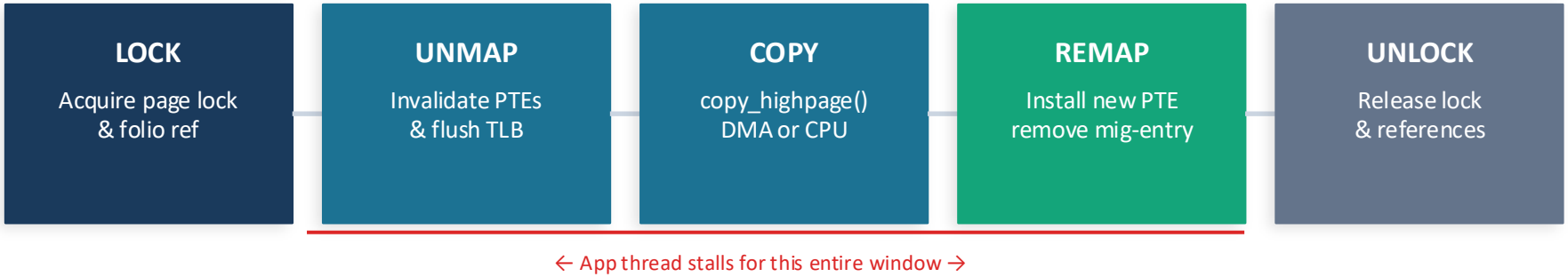
NUMA Page Migration Optimization

Quantifying and Reducing Migration Cost in Linux 6.1.4 : ARM64 & x86_64

team_name.ko
Yogit : 211207
Aayush Kumar: 230027
Cezan Vispi Damania: 230310

Problem & Motivation

NUMA migration moves memory to the node where it is accessed most, every page migration stalls application threads for the full pipeline duration. We look for optimization in these stalls.



ARM64: Hardware TLBI broadcast (~0.4 µs/page) : Copy dominates at 46%; no software IPI

x86_64: Per-page IPI shootdown (~2.6 µs/page) : Unmap already 41% quiescent, rises to 83% under load

Kernel: Instrumented via debugfs CSV ring buffer : 5 nanosecond timestamps per migration event

Proposed Optimizations: ARM64

ROCM: Read-Only Copy Migration

Targets: Full stall window • file-backed read-only pages

- Problem: faulting threads stall for Unmap+Copy+Remap even for read-only pages whose content is stable throughout
- Insight: copy can happen while source PTEs are live, no migration swap entry needed for read-only mappings
- ROCM path: copy src→dst first (no stall), then atomic PTE CMPXCHG to swap old→new translation
- Remap stage fully eliminated: remove_migration_ptes() never called; stall = PTE swap only
- Stall reduced: Unmap+Copy+Remap (~875 ns) → one TLBI round-trip (~420–462 ns)

Proposed Optimizations : x86_64

O2a Deferred Batch IPI Shootdown

Targets: Unmap stage: eliminates N per-page IPIs

- Standard: ptep_clear_flush() per page → INVLPG + flush_tlb_others() IPI; migrator blocks ~159 µs per page until all remote CPUs ACK
- 512-page batch: 512 × 159 µs = 81.5 ms IPI wait; workers interrupted 512 separate times
- O2a Phase 1: ptep_get_and_clear() clears PTE atomically, no INVLPG, no IPI; registers batch via set_tlb_ubc_flush_pending()
- O2a Phase 2: after N pages unmapped → ONE try_to_unmap_flush() → single arch_tlbbatch_flush() IPI round-trip for entire batch
- Saving: N × 159 µs → ~159 µs for entire batch (MIG_BATCH_SIZE = 32 in implementation)

O2b Address-Ordered Sort

Targets: Structural rmap walk, cache locality

- T5 analysis: structural walk cost (rmap tree + page-table walk) is 28% (683ns) of try_migrate_ns even quiescent
- Page-table nodes for nearby virtual addresses share cache lines; random VA order → ~cache miss per PTE access
- O2b: sort candidate page list by virtual address using mig_pfn_cmp + list_sort before the batch unmap loop
- Nearby VAs processed consecutively → PTE nodes stay warm in L1/L2 across the batch; fewer cold misses
- Compounds with O2a: address-ordered unmap feeds the deferred IPI batch with higher overall cache efficiency

Benchmark Suite: A to E + ROCM Comparison

A — **4KB Quiescent**

Per-page timing baseline: 512 and 2048 pages, single-threaded, quiescent. Establishes Lock/Unmap/Copy/Remap/Unlock means for both platforms.

B — **2MB THP**

2MB huge-page migration: 32 and 128 THPs. Validates density-based THP splintering hypothesis and copy-cost scaling with page size.

C — **Shared-Page Sweep**

Sweep sharing degree 1–64 across forked processes. Quantifies rmap-walk cost growth; motivates hysteresis threshold selection.

D — **Migration Downtime**

Application-visible stall measurement. Compares DMA vs CPU copy. Proves stall is bounded by migration PTE duration, not copy mechanism.

E — **Concurrent Load**

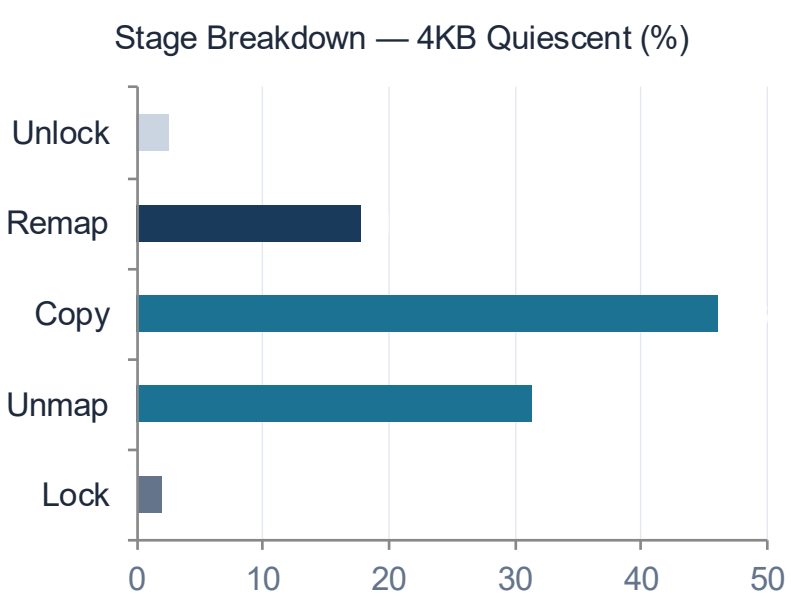
7 configs: chunk size (c1/c64/c512), thread count (t1/t4/t8), access pattern (RMW/read). Measures Disruption Factor and p999 tail latency.

R — **ROCM Comparison**

Controlled comparison: 2-worker random-read, 32 MB file-backed buffer. Only migration pipeline differs (standard vs ROCM fast-path).

ARM64 Results

ARM64 : Quiescent Baseline (Benchmarks A–D)



420 ns

Unmap/page
TLBI VAE1IS+DSB ISH

610 ns

Copy/page
copy_highpage() CPU

~251 µs

Max app downtime
Bench D

0.30 µs

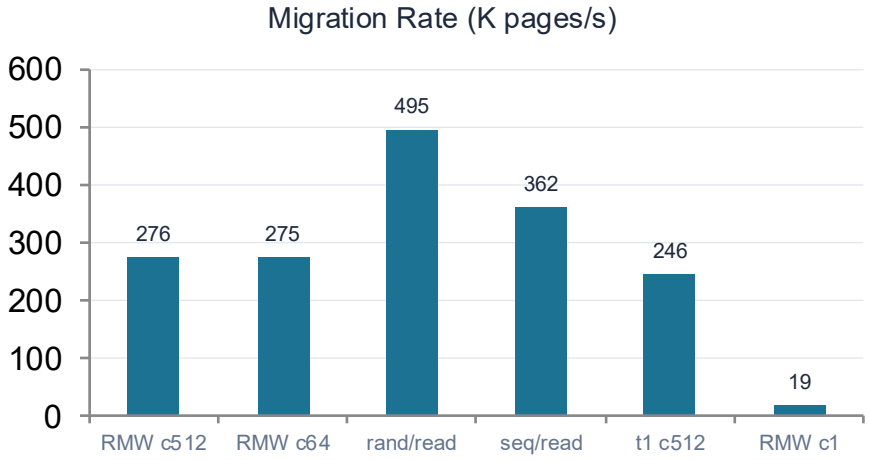
rmap walk/process
Bench C slope

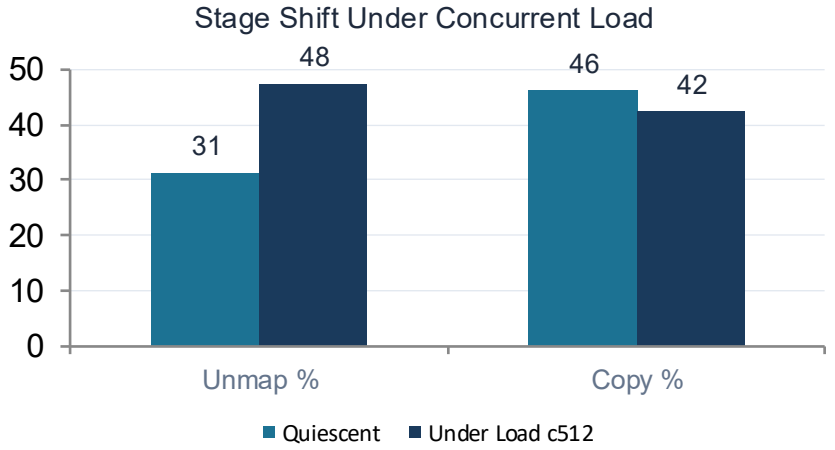
Findings

Copy dominates at 46% : TLB cost (31%) is modest quiescent → motivates ROCM for read-only pages

DMA does not reduce app downtime : stall is bounded by migration PTE duration, not copy mechanism (Bench D)

Benchmark E: Concurrent Load Results

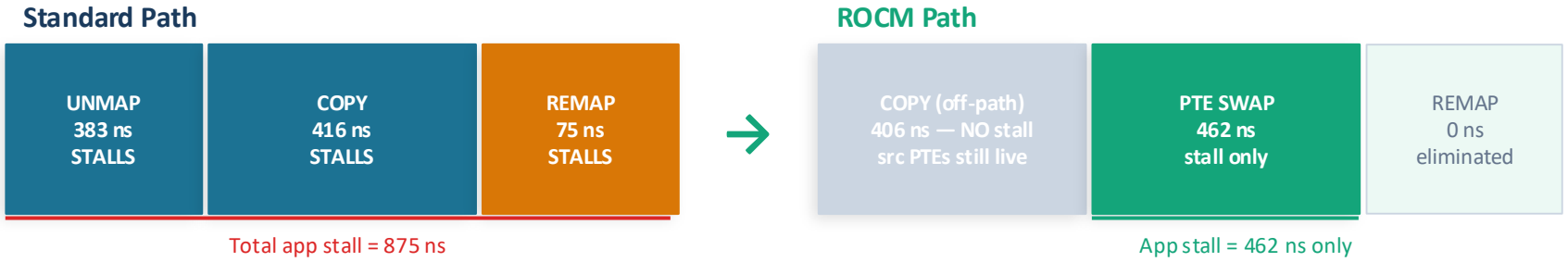




Disruption Factor & p999 Spike per Config

Config	P1 ops/s	DF (P2/P1)	p999 spike	pages/s
rand_rmw c512	84.3M	0.986	1.1×	275,577
rand_read c512	131.4M	1.053	1.1×	495,208
seq_read c512	130.4M	1.085	0.9×	361,555
rand_rmw c1	80.0M	0.948	27.5×	19,324
rand_rmw t1 c512	38.5M	1.187	0.9×	245,620

ROCM: Measured Results

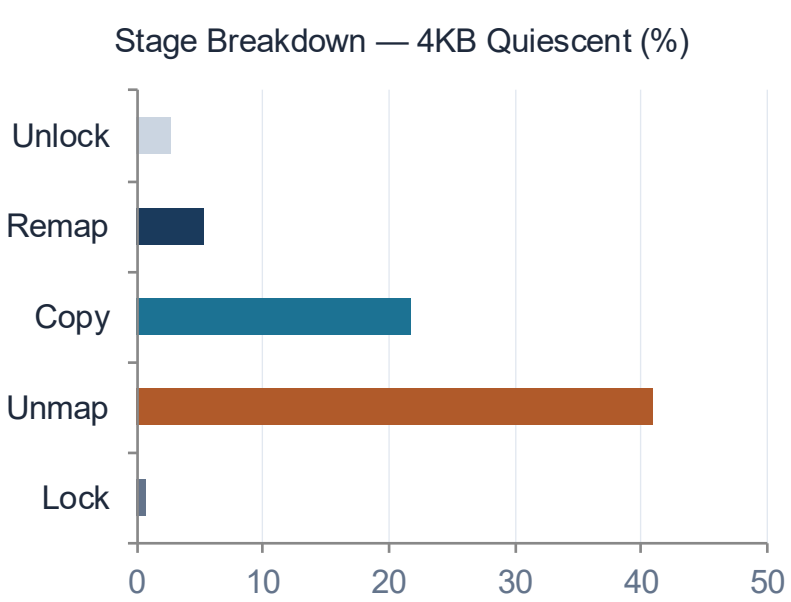


Measured Results: Controlled Comparison (2 workers, random read, 32 MB file-backed)

Metric	Standard	ROCM	Improvement
App stall window (primary metric)	875 ns	462 ns	1.89× reduction ✓
Remap stage	75 ns	0 ns	Fully eliminated ✓
Migration rate	46,704 pg/s	84,154 pg/s	1.80× faster ✓
Mean stall duration (Bench E)	54,869 ns	26,729 ns	2.05× shorter ✓
rocm_path=1 records	0 / 8192	8192 / 8192	100% pages qualify ✓
Disruption Factor	1.020	1.077	+5.6% improvement

x86_64 Results

x86_64: Quiescent Baseline & the IPI Bottleneck



~2.6 µs

Unmap/page
INVLPG + IPI shootdown

1,380 ns

Copy/page
copy_highpage() CPU

~702 µs

Max app downtime
Bench D

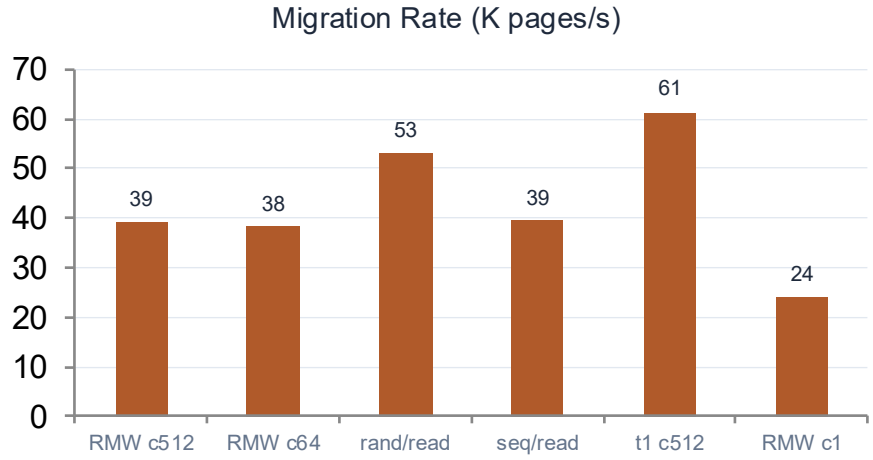
159 µs

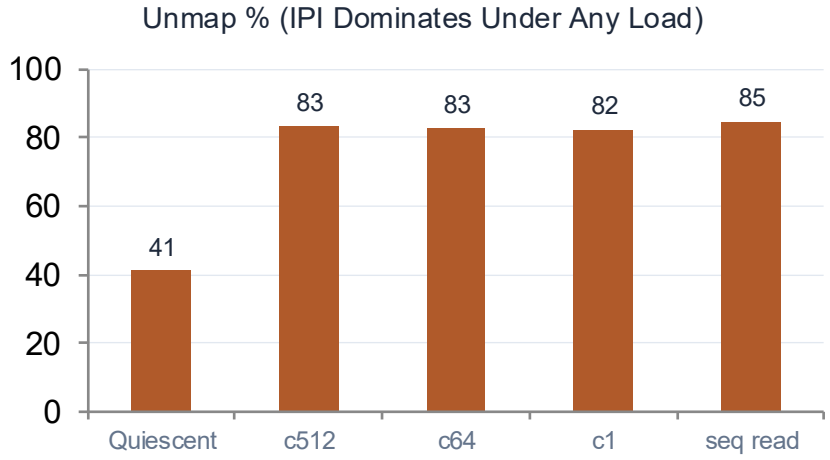
Per-page IPI wait
under load

Per-Page IPI Shootdown Sequence (ptep_clear_flush)

1. **LOCK CMPXCHG** : atomically clear PTE (~5 ns); no IPI
2. **INVLPG** : flush local CPU TLB entry (~5 cycles)
3. **flush_tlb_others() IPI** → interrupt ALL remote CPUs → block until every CPU ACKs (~159 µs under load)
4. **Repeat per page** : 512-page batch = 512 separate IPI round-trips = 81.5 ms total IPI wait

x86_64 — Benchmark E Results





Disruption Factor & p999 Spike per Config

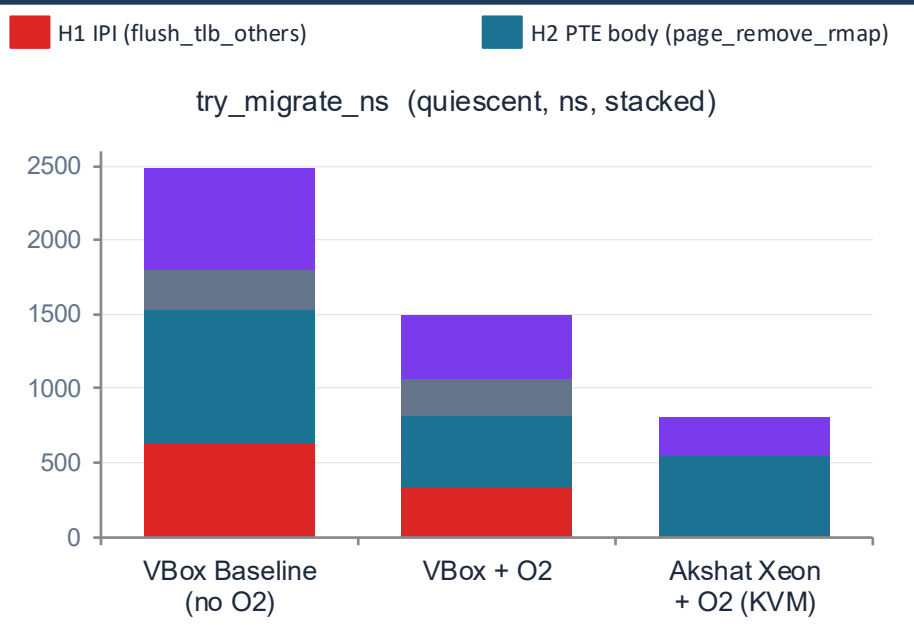
Config	P1 ops/s	DF (P2/P1)	p999 spike	pages/s
rand_rmw c512	27.8M	0.133	13.8×	39,118
rand_read c512	66.8M	0.248	14.7×	53,083
seq_read c512	80.3M	0.098	15.8×	39,422
rand_rmw c1	28.6M	0.135	23.5×	24,072
rand_rmw t1 c512	18.2M	0.193	11.9×	61,410

Hypothesis Testing: Isolating the Bottleneck

Five hypotheses tested to identify the true cause of Unmap dominance before proposing O2:

H1	**IPI shootdown dominates : Unmap scales linearly with # online CPUs** *Test: T1 (Varies number of CPU) + T5 (rmap sub-timing)* T5: ipi_wait_ns = 98.5% of try_migrate_ns under load. T1: R^2=0.85+ on bare metal, each CPU offline reduces unmap_ns proportionally. → Motivates O2a.	CONFIRMED	→ O2a
H2	**PTE spinlock (pte_lockptr) contention from concurrent workers** *Test: T5: ptl_wait_ns isolation under Bench E concurrent load* ptl_wait_ns < 2% quiescent, modest rise under load only. Workers are blocked by migration PTE before reaching ptl, not the bottleneck.	RULED OUT	no action
H3	**rwsem contention from kswapd, burst outlier pattern in Unmap tail** *Test: T3: outlier burst pattern + vmstat kswapd cross-reference* Outliers < 2% of records, randomly distributed. kswapd activity does not correlate with Unmap spikes. Burst pattern absent.	RULED OUT	no action
H4	**Cache thrash, rmap/page-table nodes cold after 32+ migrations in sequence** *Test: T2: try_migrate_ns vs sequential record number quintile* Q5/Q1 < 1, no monotonic rise with migration count. Page-table nodes stay warm within a run. Cache thrash not causing Unmap growth.	RULED OUT	no action
H5	**Structural walk cost (rmap tree + page-table walk) is secondary bottleneck** *Test: T5: ipi_wait_ns vs outside-ipi fraction of try_migrate_ns* 28% of try_migrate_ns is structural walk cost even quiescent. Survives O2a fix. → Motivates O2b (address-ordered sort) to reduce PTE cache misses.	CONFIRMED	→ O2b

Unmap Sub-Stage Breakdown: Baseline vs Optimized



Under Load (VBox Baseline): try_migrate_ns = 141,714 ns — 57× quiescent

H1 IPI	H2 PTE body	Structural walk
139,538 ns	1,291 ns	885 ns
98.5%	0.9%	0.6%

Detailed Sub-Stage Numbers — Quiescent (T5 Breakdown)

Component	VBox Baseline	VBox + O2	Akshat O2 (KVM)
H1 IPI (flush_tlb_others)	632 ns 25.4%	335 ns 22.5%	~110 ns amortised
H2 PTE body	894 ns 36.0%	480 ns 32.2%	~539 ns 66.6%
H3 rwsem (down_read)	277 ns 11.1%	244 ns 16.4%	0 ns — 0% (KVM absorbs)
Structural walk (H5 target)	683 ns 27.5%	429 ns 28.9%	270 ns 33.4%
try_migrate_ns total	**2,486 ns**	**1,488 ns** **1.67×↑**	**919 ns** **2.73×↑**
unmap_ns total	**2,609 ns**	**1,642 ns** **1.59×↑**	—

O2a (IPI): 632→335 ns (1.89×). Under load it explodes to 139,538 ns (98.5%) — O2a collapses 512 IPIs to 1.

O2b (walk): 683→429 ns (1.59×) on VBox; 683→270 ns (2.02×) on Akshat. Larger L1/L2 amplifies PMD locality.

KVM effect: IPI: VM exit overhead only (~100 ns amortised, not a real IPI round-trip). rwsem: cache-hot, effectively zero. Walk alone accounts for 33.4%. O2b validated independently of O2a.

x86_64 : O2 Optimization Results

Bench A: Quiescent Stage Timing : Before vs After O2

Metric	Baseline	After O2	Gain
try_migrate_ns median	2,486 ns	1,488 ns	1.67× ✓
ipi_wait_ns (O2a target)	632 ns (25.4%)	335 ns (22.5%)	1.89× ✓
Structural walk (O2b target)	683 ns (27.5%)	429 ns (28.9%)	1.59× ✓
PTE body work	894 ns (36.0%)	480 ns (32.2%)	1.86× ✓
unmap_ns total	2,609 ns	1,642 ns	1.59× ✓

Bench E: Disruption Factor : O2 vs Baseline

Machine	DF (O2)	p999	vs Baseline
Baseline (no O2)	0.133	13.8×	reference
VirtualBox i5 (real IPI)	0.767	41,858 ns	5.8× better DF ✓
Akshat Xeon (KVM)	1.093	586 ns	Net-beneficial ✓
Cezan i7 (KVM)	1.098	838 ns	Net-beneficial ✓

Bench E: All Configs Under Load : Disruption Factor & Phase 3 Recovery After O2

Config	Baseline DF	Akshat DF	Akshat P3	Cezan DF	Cezan P3	Agreement
rand_rmw t4 c512 (primary)	0.133	1.093	1.194	1.098	1.159	Both >1 ✓✓
rand_rmw t1 c512	0.193	1.513	1.598	1.001	1.009	Both >1 ✓✓
rand_rmw t8 c512	—	1.094	1.130	1.011	1.032	Both >1 ✓✓
seq_read t4 c512	0.098	0.878	0.913	1.052	1.098	Mixed
rand_read t4 c512	0.248	0.865	0.814	1.033	1.010	Mixed

Key Findings & Conclusions

ARM64: Batched TLB: load-dependent Unmap confirmed

Unmap rises 31%→47% under concurrent load; N DSB ISH stalls collapse to 1. ~194 µs saved per 512-page batch. ARMv8.4+ range instructions are the implementation target.

ARM64: ROCM delivers 1.89× stall reduction for read-only pages

875 ns → 462 ns app stall. Remap stage fully eliminated. Migration rate 1.80× faster. Mean stall 2.05× shorter. Bounded by one TLBI round-trip — near-optimal.

x86: H1 confirmed (T1+T5) : IPI shootdown is 98.5% of Unmap under load

H2/H3/H4 ruled out by systematic testing. Unmap reaches 83–85% under load. Workers reduced to 10–25% baseline throughput (DF=0.10–0.25) from continuous IPI interruption.

x86: O2a (Deferred Batch IPI) + O2b (Address-Ordered Sort) designed

O2a: N per-page IPIs → 1 batch IPI; ~81.5 ms → ~159 µs for 512 pages. O2b: VA-ordered sort reduces 28% structural walk cache-miss cost identified by T5.

Both: chunk ≥ 64 is the minimum viable batch size on both platforms

chunk=1 → 23–27× p999 spikes and 6–14× lower migration rate. c64 and c512 produce identical rates — kernel internal batching saturates below chunk=64.